Nithin Gowtham Saravanan

Linux Kernel: Built-in Modules

TABLE OF CONTENTS

Nithin Gowtham Saravanan

Introduction

The objective of this laboratory exercise was to understand the integration of device drivers as **built-in modules within the Linux kernel** in an embedded Linux environment on the Zybo Z7-10 board using PetaLinux. Unlike the previous lab where the multiplier driver was implemented as a **loadable kernel module (LKM)**, this lab focuses on embedding the driver directly into the kernel so that it is automatically initialized during system boot.

This lab also introduces kernel configuration using **Kconfig and menuconfig**, allowing selective inclusion and exclusion of kernel components. Additionally, unnecessary drivers such as networking, multimedia, and sound support were disabled to generate a **reduced-size kernel image**, improving system efficiency.

This exercise provided practical exposure to kernel customization, built-in driver integration, and kernel size optimization.

Procedure

1. PetaLinux Setup and Project Preparation

1. A PetaLinux installation directory was created under /tmp for the user workspace:
   cd /tmp
   mkdir petalinux_nithingowtham25
2. The PetaLinux installer was copied from the shared lab directory:
   cd petalinux_nithingowtham25
   cp /mnt/lab_files/ECEN449/petalinux-v2023.1-05012318-installer.run .
3. A directory was created for installation and the installer was executed:
   mkdir petalinux_install
   chmod 755 ./petalinux-v2023.1-05012318-installer.run
   ./petalinux-v2023.1-05012318-installer.run --dir petalinux_install --platform "arm"
4. After installation, environment variables were loaded:
   source petalinux_install/settings.sh
5. A new PetaLinux project was created using the Zynq template:
   petalinux-create -t project --template zynq -n linux_boot_nithingowtham25 --tmpdir /tmp/petalinux_nithingowtham25/tmp_dir
6. The hardware configuration was updated using the exported Vivado design:
   cd linux_boot_nithingowtham25
   petalinux-config --get-hw-description=/home/grads/n/nithingowtham25/Spring2026/ECEN749/lab4_new/linux_boot_wrapper.xsa --silentconfig

2. Kernel Configuration

7. The kernel configuration menu was accessed using:
   petalinux-config -c kernel

8. The menuconfig interface was explored to understand kernel options and dependencies.

3. Adding Multiplier Driver to Kernel Source

9. The Linux source code was extracted into the PetaLinux project directory under:
   components/ext_sources/
10. A new directory was created for the driver: drivers/multiplier_driver/
11. The multiplier driver source files (multiplier.c and headers) were copied into this directory (shown in **Appendix I**).

4. Creating Build Configuration Files

12. A **Makefile** was created in the driver directory with the following entry:
    obj-$(CONFIG_MULTIPLIER_DRIVER) += multiplier.o
13. A **Kconfig** file was created with:
    - Tristate configuration (Y/M/N)
    - Dependency on ARM architecture
    - Default enabled for ARM systems
14. The driver was linked to the kernel build system:
    - Added entry in drivers/Makefile
    - Added source in drivers/Kconfig

5. Linking External Kernel Source

15. The modified Linux source was linked to the PetaLinux project:
    petalinux-config
16. External kernel source path was provided under:
    Linux Components Selection → linux-kernel → ext-local-src

6. Enabling Built-in Driver

17. Kernel configuration was reopened:
    petalinux-config -c kernel
18. The multiplier driver was visible under the **Device Drivers** menu in menuconfig, confirming successful integration into the kernel configuration (shown in **Appendix A**).
19. The multiplier driver was enabled as: Device Drivers → multiplier driver → Built-in (Y)

7. Kernel Build and Deployment

20. Kernel version check was skipped by modifying the .bb file.
21. The system was built:
    petalinux-build

(Build success shown in **Appendix B**)

22. The generated image.ub size was observed as **18 MB** (shown in **Appendix C**).
23. The generated image.ub was copied to the SD card.
24. The system was booted on the Zybo Z7-10 board, and the boot sequence was verified (shown in **Appendix D**).

8. Verification of Built-in Driver

25. During boot, the initialization message from the multiplier driver was observed (**Appendix D**).
26. Since the driver is built into the kernel, no insmod command was required.
27. The device node was created using:
    mknod /dev/multiplier c 244 0
28. The user application (devtest) was executed:
    ./devtest
    Execution shown in **Appendix E**)
29. Correct multiplication results were verified using devtest output (shown in **Appendix F**).

9. Kernel Optimization

30. The following drivers were disabled using menuconfig (shown in **Appendix G**):
    - Network device support
    - Multimedia support
    - Sound card support
31. The kernel was rebuilt: petalinux-build
32. The reduced kernel image size of **16.5 MB** was observed and compared with the initial size (shown in **Appendix H**).

(The complete working of the kernel was demonstrated to the TA)

Results

The embedded Linux system successfully booted with the multiplier driver integrated as a **built-in kernel module**, as shown in **Appendix D**. The driver initialization message appeared during boot, confirming that the driver was automatically loaded by the kernel.

The kernel build was completed successfully (**Appendix B**), and the initial kernel image size **(18MB)** was recorded (**Appendix C**). After disabling unnecessary drivers, the kernel was rebuilt, resulting in a significantly reduced image size of **16.5MB** (**Appendix H**).

The user application executed successfully (**Appendix E**), and the devtest output (**Appendix F**) confirmed correct multiplication functionality. The results matched expected values for all tested inputs.

The following key observations were made:

- The driver was automatically initialized during boot without manual insertion
- Kernel configuration successfully controlled inclusion of drivers
- System functionality remained intact after removing unused drivers
- Kernel image size was significantly reduced after optimization
- Built-in modules improved boot-time readiness of the driver

The successful operation of the built-in driver and optimized kernel was demonstrated to the TA.

Conclusion

This laboratory exercise successfully demonstrated the integration of a device driver as a **built-in** component of the Linux kernel. By modifying the kernel source, configuring Kconfig entries, and enabling the driver through menuconfig, a deeper understanding of kernel-level design and driver integration was achieved.

The multiplier driver was automatically initialized during system boot, eliminating the need for manual insertion and improving system readiness. This highlights the advantage of built-in modules in embedded systems where reliability and minimal user intervention are important.

Additionally, **kernel optimization** was performed by disabling unnecessary drivers such as network, multimedia, and sound support. This resulted in a noticeable **reduction in kernel image size**, demonstrating the importance of customizing the kernel based on system requirements.

Overall, this lab reinforced concepts of kernel configuration, built-in driver integration, and system optimization, while also emphasizing the trade-offs between flexibility and efficiency in embedded Linux systems.

Knowledge Transfer

a) What are the advantages and disadvantages of loadable kernel modules and built-in modules?

BUILT-IN MODULES

Advantages:

- Automatically loaded during system boot
- No need for manual insertion (insmod)
- Faster availability during runtime

Disadvantages:

- Increases kernel size
- Cannot be removed or modified without recompiling the kernel
- Less flexible compared to modular approach

LOADABLE KERNEL MODULES (LKM)

Advantages:

- Can be dynamically loaded and unloaded
- Easier debugging and updates without rebuilding kernel
- Keeps base kernel smaller and more modular

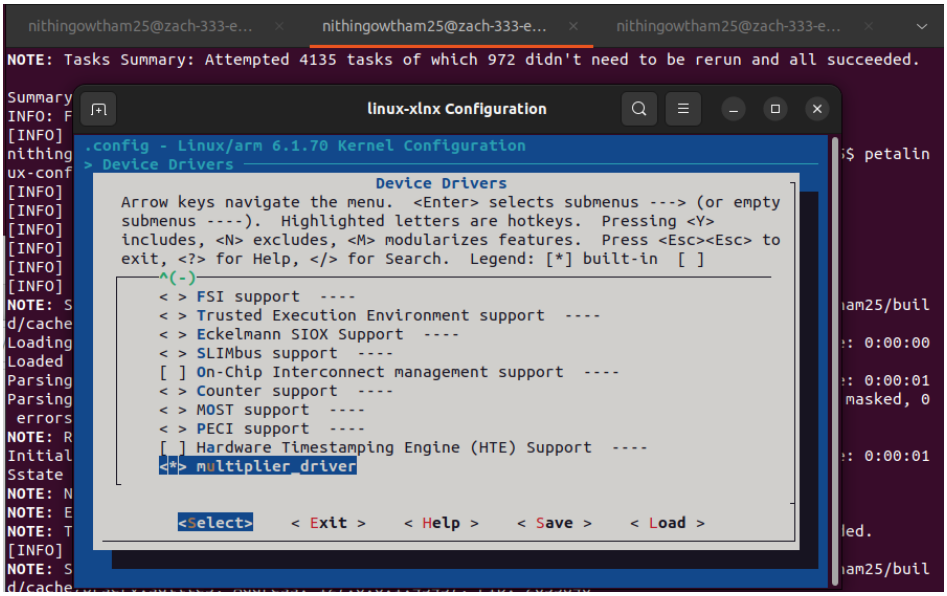
Disadvantages:

- Requires manual loading
- Slight runtime overhead due to dynamic insertion
- Dependency management required between modules

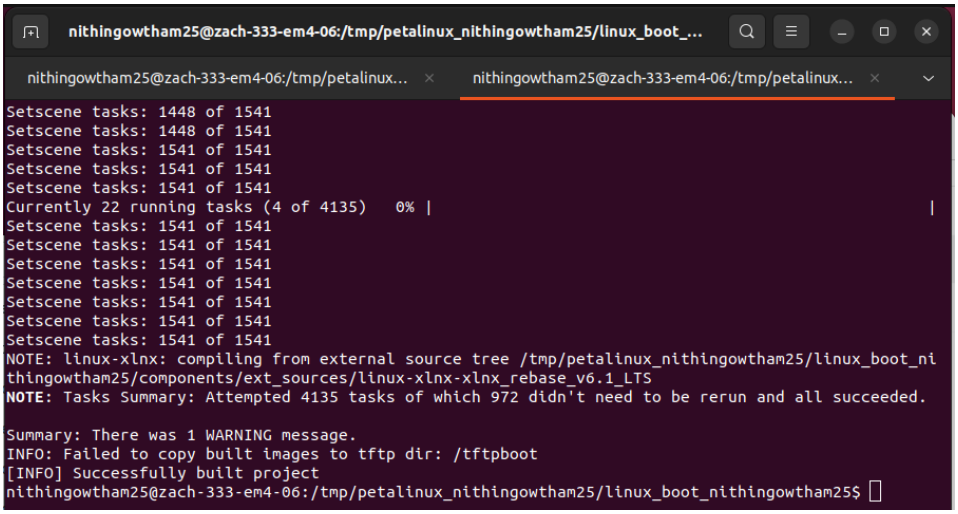
Nithin Gowtham Saravanan

Appendix

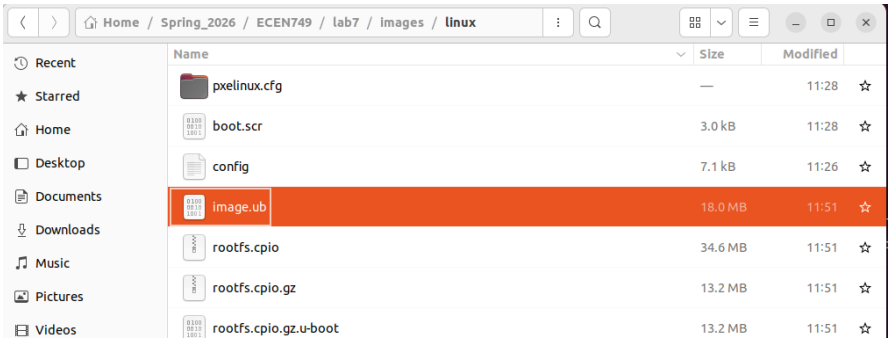
APPENDIX A – Multiplier Driver in Kernel Config



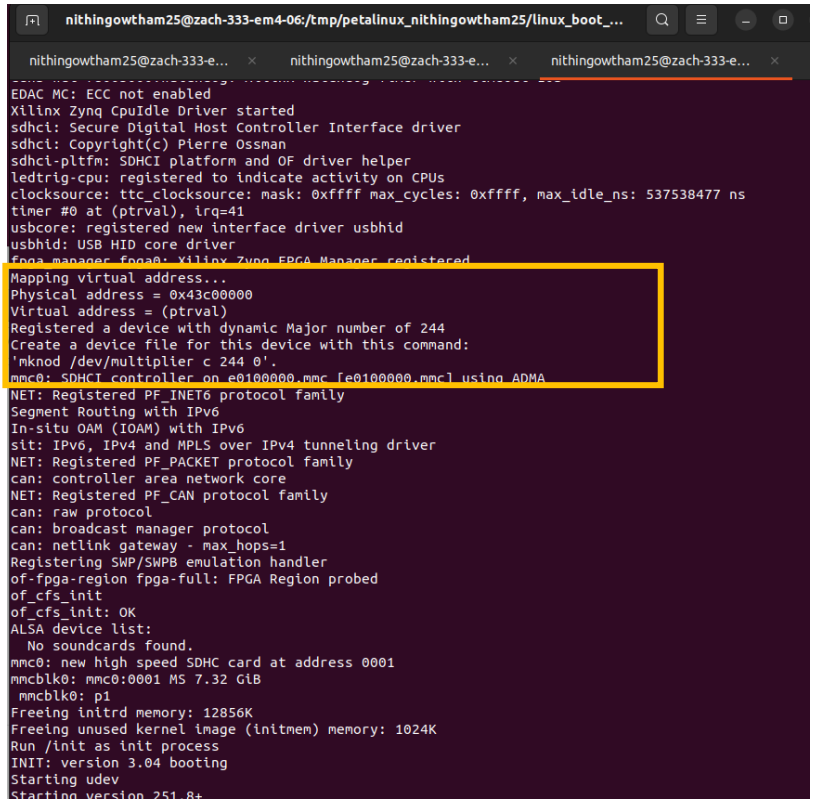
APPENDIX B – Initial Build Success



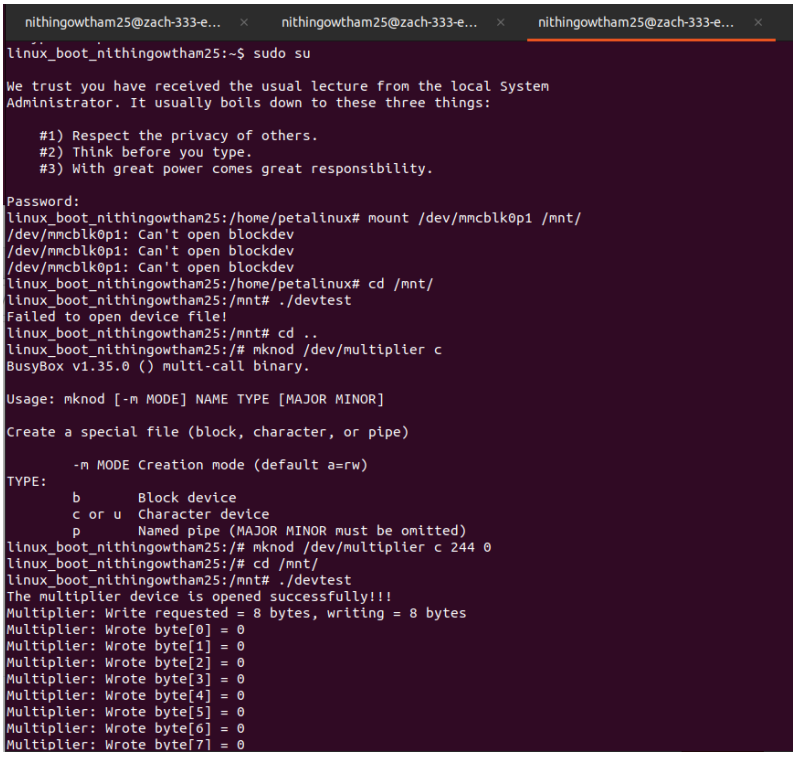
APPENDIX C – Initial Image Size



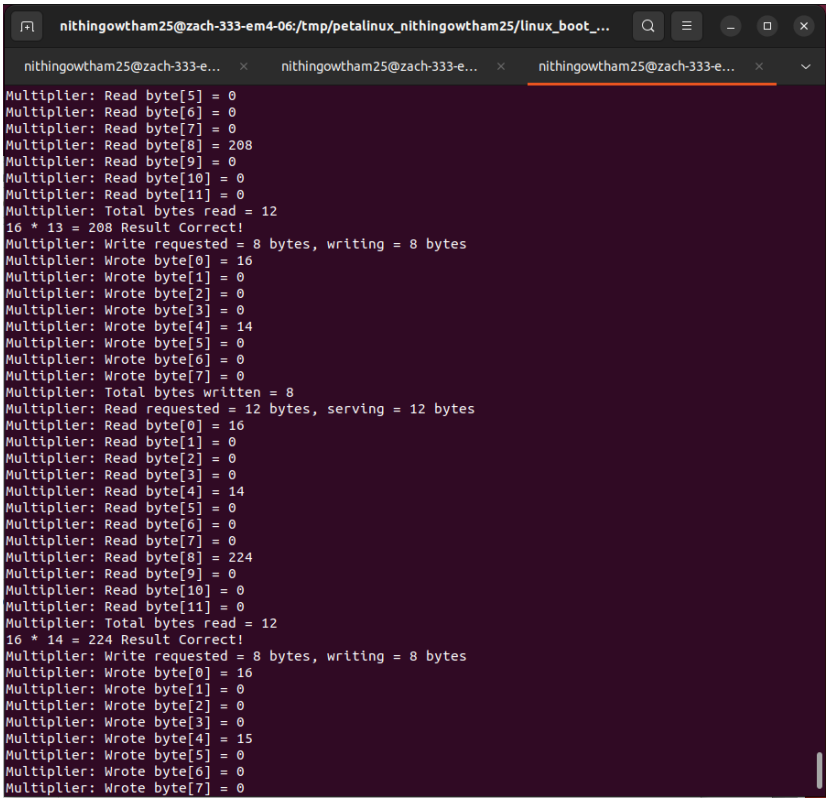
APPENDIX D – Boot Initialization Message



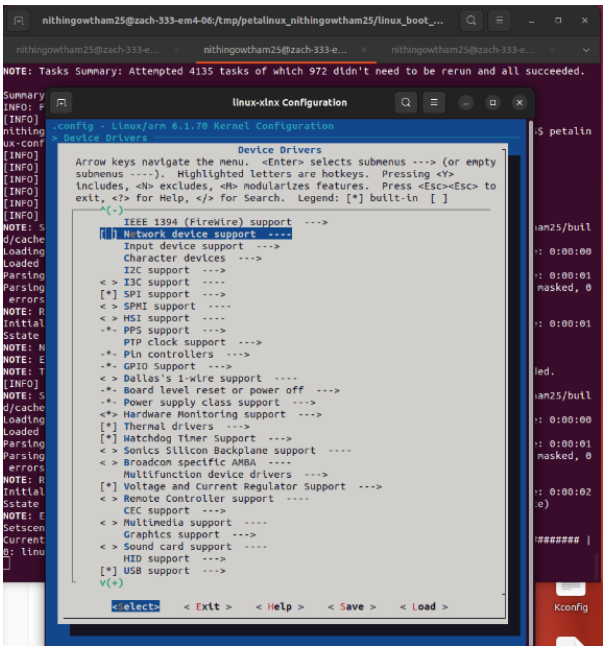
APPENDIX E – User Application Running



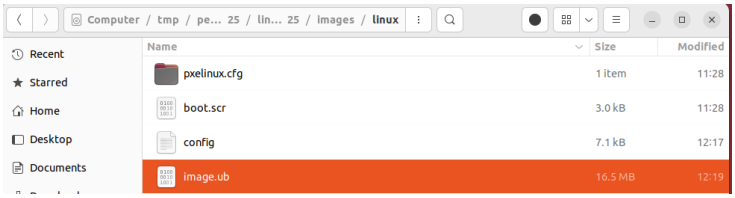
APPENDIX F – devtest output matched



APPENDIX G – Excluded Drivers

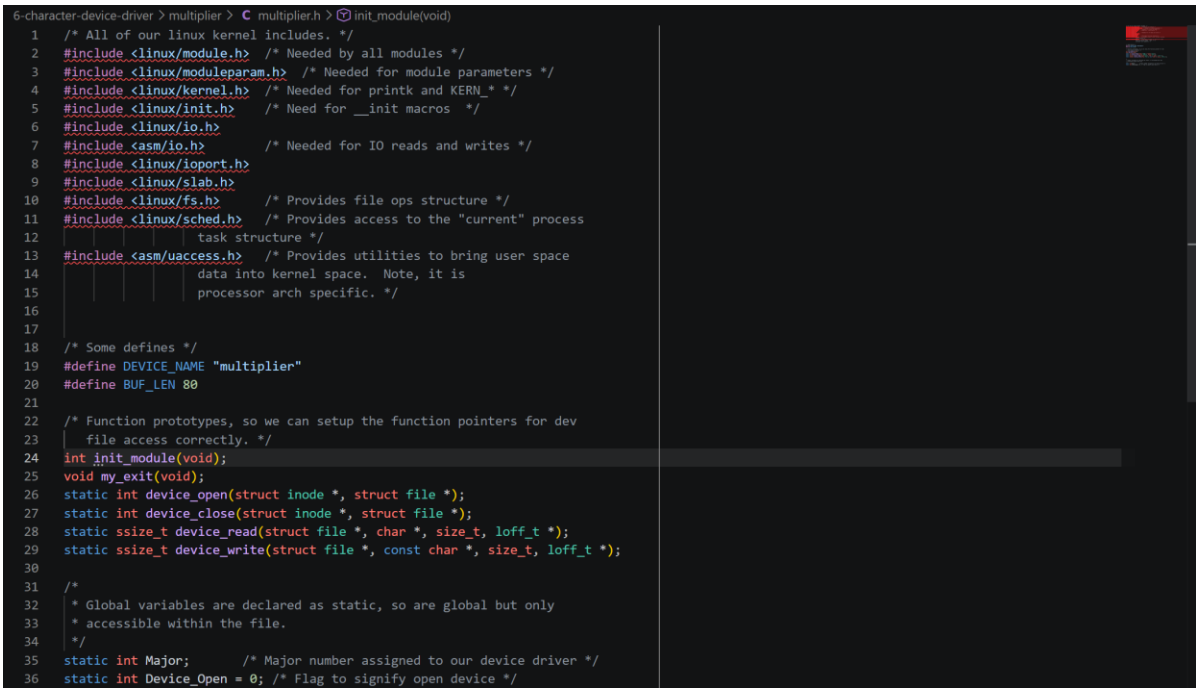


APPENDIX H – Reduced Image Size



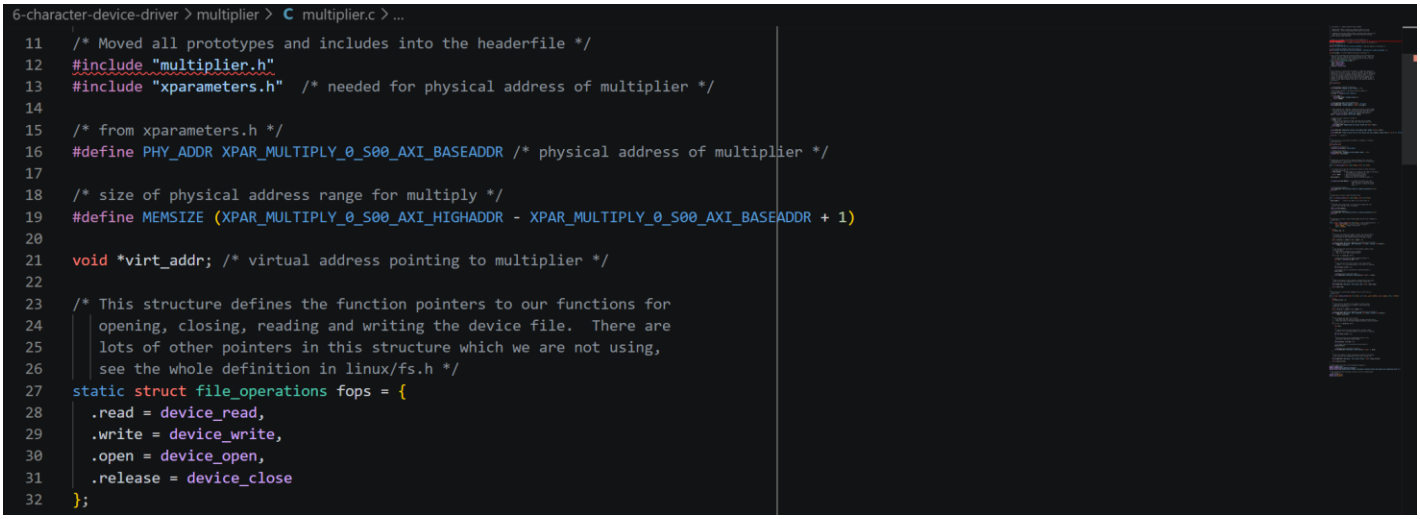
APPENDIX I (Source Code)

- **multiplier.h**



```
6-character-device-driver > multiplier > C multiplier.h > init_module(void)
1  /* All of our linux kernel includes. */
2  #include <linux/module.h>  /* Needed by all modules */
3  #include <linux/moduleparam.h>  /* Needed for module parameters */
4  #include <linux/kernel.h>  /* Needed for printk and KERN_* */
5  #include <linux/init.h>    /* Need for __init macros  */
6  #include <linux/io.h>
7  #include <asm/io.h>        /* Needed for IO reads and writes */
8  #include <linux/ioport.h>
9  #include <linux/slab.h>
10 #include <linux/fs.h>      /* Provides file ops structure */
11 #include <linux/sched.h>   /* Provides access to the "current" process
12                               task structure */
13 #include <asm/uaccess.h>   /* Provides utilities to bring user space
14                               data into kernel space.  Note, it is
15                               processor arch specific. */
16
17
18 /* Some defines */
19 #define DEVICE_NAME "multiplier"
20 #define BUF_LEN 80
21
22 /* Function prototypes, so we can setup the function pointers for dev
23    file access correctly. */
24 int init_module(void);
25 void my_exit(void);
26 static int device_open(struct inode *, struct file *);
27 static int device_close(struct inode *, struct file *);
28 static ssize_t device_read(struct file *, char *, size_t, loff_t *);
29 static ssize_t device_write(struct file *, const char *, size_t, loff_t *);
30
31 /*
32  * Global variables are declared as static, so are global but only
33  * accessible within the file.
34  */
35 static int Major;       /* Major number assigned to our device driver */
36 static int Device_Open = 0; /* Flag to signify open device */
```

- **multiplier.c**



```
6-character-device-driver > multiplier > C multiplier.c > ...
11 /* Moved all prototypes and includes into the headerfile */
12 #include "multiplier.h"
13 #include "xparameters.h"  /* needed for physical address of multiplier */
14
15 /* from xparameters.h */
16 #define PHY_ADDR XPAR_MULTIPLY_0_S00_AXI_BASEADDR /* physical address of multiplier */
17
18 /* size of physical address range for multiply */
19 #define MEMSIZE (XPAR_MULTIPLY_0_S00_AXI_HIGHADDR - XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1)
20
21 void *virt_addr; /* virtual address pointing to multiplier */
22
23 /* This structure defines the function pointers to our functions for
24    opening, closing, reading and writing the device file.  There are
25    lots of other pointers in this structure which we are not using,
26    see the whole definition in linux/fs.h */
27 static struct file_operations fops = {
28   .read = device_read,
29   .write = device_write,
30   .open = device_open,
31   .release = device_close
32 };
```

Nithin Gowtham Saravanan

```c
int my_init(void)
{

  /* Linux kernel's version of printf */
  printk(KERN_INFO "Mapping virtual address...\n");

  /* map virtual address to multiplier physical address */
  /* use ioremap */
  virt_addr = ioremap(PHY_ADDR, MEMSIZE);

  if (!virt_addr) {
      printk(KERN_ALERT "ioremap failed!\n");
      return -ENOMEM;
  }

  /* print physical and virtual addresses */
  printk(KERN_INFO "Physical address = %#x\n", PHY_ADDR);
  printk(KERN_INFO "Virtual address = %p\n", virt_addr);


  /* This function call registers a device and returns a major number
      associated with it.  Be wary, the device file could be accessed
      as soon as you register it, make sure anything you need (ie
      buffers ect) are setup _BEFORE_ you register the device.*/
  Major = register_chrdev(0, DEVICE_NAME, &fops);


  /* Negative values indicate a problem */
  if (Major < 0) {
    /* Make sure you release any other resources you've already
        grabbed if you get here so you don't leave the kernel in a
        broken state. */
    printk(KERN_ALERT "Registering char device failed with %d\n", Major);
    return Major;
  }

  printk(KERN_INFO "Registered a device with dynamic Major number of %d\n", Major);

  printk(KERN_INFO "Create a device file for this device with this command:\n'mknod /dev/%s c %d 0'.\n", DEVICE_NAME, Major);

  return 0;   /* success */
}

/*
 * This function is called when the module is unloaded, it releases
 * the device file.
 */
void my_exit(void)
{
  /* Unregister the device */
  unregister_chrdev(Major, DEVICE_NAME);

  /* Unmap virtual memory */
  printk(KERN_ALERT "unmapping virtual address space....\n");
  iounmap((void *)virt_addr);

}
```

```c
static int device_open(struct inode *inode, struct file *file)
{

  /* In these case we are only allowing one process to hold the device
      file open at a time. */
  if (Device_Open)    /* Device_Open is my flag for the usage of the device
                          file (definied in multiplier.h)  */
    return -EBUSY;    /* Failure to open device is given
                          back to the userland program. */
  Device_Open++;      /* Keeping the count of the device opens. */


  try_module_get(THIS_MODULE);  /* increment the module use count
                                   (make sure this is accurate or you
                                   won't be able to remove the module
                                   later. */

  /* Print kernel message */
  printk(KERN_INFO "The multiplier device is opened successfully!!!\n");
  return 0;
}

/*
 * Called when a process closes the device file.
 */
static int device_close(struct inode *inode, struct file *file)
{
  Device_Open--;    /* We're now ready for our next caller */

  /*
   * Decrement the usage count, or else once you opened the file,
   * you'll never get get rid of the module.
   */
  module_put(THIS_MODULE);

  /* Print kernel message */
  printk(KERN_INFO "The multiplier device is closed successfully!!!\n");
  return 0;
}
```

Nithin Gowtham Saravanan

```c
static ssize_t device_read(struct file *filp, /* see include/linux/fs.h   */
                           char __user *buffer, /* buffer to fill with data */
                           size_t length, /* length of the buffer     */
                           loff_t * offset)
{
    int i;
    int bytes_read = 0;

    /*
     * The user may request any number of bytes, but the multiplier
     * peripheral only supports addresses from byte 0 to byte 11.
     * So we compute a safe length that does not exceed 12 bytes.
     */
    size_t valid_len = (length <= 12) ? length : 12;

    /* Print how many bytes were requested vs how many will be served */
    printk(KERN_INFO "Multiplier: Read requested = %zu bytes, serving = %zu bytes\n",
           length, valid_len);

    /*
     * Loop through each valid byte in the peripheral address space.
     * For each byte:
     *  - Read from the hardware using ioread32()
     *  - Copy it to user space using put_user()
     */
    for (i = 0; i < valid_len; i++) {

        /* Read one byte from the memory-mapped peripheral */
        char data = ioread32(virt_addr + i);

        /*
         * Copy the byte from kernel space to user space buffer.
         * 'buffer' is a user-space pointer, so put_user is required.
         */
        put_user(data, buffer + i);

        /* Increment count of successfully transferred bytes */
        bytes_read++;

        printk(KERN_INFO "Multiplier: Read byte[%d] = %d\n", i, data);
    }

    /*
     * Return the number of bytes actually transferred to user space.
     * This satisfies the requirement of the read() system call.
     */
    printk(KERN_INFO "Multiplier: Total bytes read = %d\n", bytes_read);

    return bytes_read;
}
```

```c
static ssize_t device_write(struct file *file, const char __user * buffer, size_t length, loff_t * offset)
{
    int i;
    int bytes_written = 0;

    /*
     * The multiplier peripheral only supports writes to byte
     * addresses 0 through 7. So we compute a safe length that
     * does not exceed 8 bytes.
     */
    size_t valid_len = (length <= 8) ? length : 8;

    /* Print how many bytes were requested vs how many will be written */
    printk(KERN_INFO "Multiplier: Write requested = %zu bytes, writing = %zu bytes\n",
           length, valid_len);

    /*
     * Loop through each byte to be written:
     *  - Copy data from user space to kernel variable using get_user()
     *  - Write that byte to the memory-mapped peripheral using iowrite32()
     */
    for (i = 0; i < valid_len; i++) {

        char data;

        /*
         * Copy one byte from user space buffer to kernel space.
         * 'buffer' is a user-space pointer, so get_user is required.
         */
        get_user(data, buffer + i);

        /*
         * Write the byte to the corresponding offset in the
         * multiplier peripheral address space.
         */
        iowrite32(data, virt_addr + i);

        /* Increment count of successfully written bytes */
        bytes_written++;

        /* Debug print for each byte written */
        printk(KERN_INFO "Multiplier: Wrote byte[%d] = %d\n", i, data);
    }

    /*
     * Return the number of bytes successfully written to the device.
     * This satisfies the requirement of the write() system call.
     */
    printk(KERN_INFO "Multiplier: Total bytes written = %d\n", bytes_written);

    return bytes_written;
}

/* These define info that can be displayed by modinfo */
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Nithin Gowtham Saravanan");
MODULE_DESCRIPTION("Module which creates a multiplier character device and allows user interaction with it");

/* Here we define which functions we want to use for initialization
   and cleanup */
module_init(my_init);
module_exit(my_exit);
```